

AI-Powered Clinical Classification System

Integrated Project Report: Anemia, Diabetes, and Leukemia Prediction

Project repository	ai-powered-clinic-classification
Main application	web_app.py

This report explains the clinical problem, older methods, the proposed AI solution, the theoretical methods, the programming architecture, the user workflow, evaluation strategy, explainability, limitations, and future improvements.

The system combines two structured-data machine learning pipelines for anemia and diabetes with a deep learning image pipeline for leukemia. It exposes all models through one local web application.

1. Executive Summary

The project implements an AI-powered clinical classification platform that supports three prediction tasks: anemia classification from blood test markers, diabetes status classification from health indicators, and leukemia classification from microscopic blood smear images.

The anemia and diabetes modules use tabular machine learning pipelines. The leukemia module uses deep learning image models, including a blood-smear validator, EfficientNet-B5, EfficientNet-B4, test-time augmentation, probability ensembling, and Grad-CAM visual explanations.

The main contribution is the integration of separate clinical prediction tasks into one local browser-based workflow. A user can select a disease model, enter values manually, upload CSV files, extract CBC markers from report images, or analyze blood smear images with visual model focus maps.

2. Clinical Problem and Motivation

Clinical classification is a recurring healthcare task. Physicians interpret laboratory markers, patient indicators, or medical images to decide whether a patient is healthy, at risk, or affected by a disease. Manual interpretation can be slow, inconsistent, and sensitive to missing data.

2.1 Anemia

Anemia may be related to iron deficiency, folate deficiency, vitamin B12 deficiency, or hemoglobin abnormalities. Complete blood count features and nutritional biomarkers contain useful signals, but combining them consistently is difficult in manual workflows.

2.2 Diabetes

Diabetes and prediabetes are associated with demographic, lifestyle, and health indicators. Early screening can help reduce complications, but health-indicator datasets contain many variables and imbalanced classes.

2.3 Leukemia

Leukemia detection from microscopic blood images requires specialist visual inspection of cell morphology. Automated image analysis can support screening and triage, especially when it also explains which image regions influenced the decision.

3. Traditional and Older Approaches

Older approaches include manual threshold rules, spreadsheet comparison, isolated laboratory reports, and direct microscopy by specialists. These methods remain clinically important but have limitations when many features interact or when large volumes of cases

must be screened.

- Manual threshold rules are easy to understand but cannot model complex nonlinear interactions between biomarkers.
- Paper or spreadsheet workflows are vulnerable to manual entry errors and do not provide reusable prediction pipelines.
- Manual microscopy is expert-driven and essential, but it is time-consuming and may vary between observers.
- Single-purpose tools force the user to switch between applications and rarely provide a unified workflow for multiple diseases.

4. Proposed Solution

The proposed solution is a local clinical decision-support web application. The backend hosts multiple models behind a common API, while the frontend dynamically adapts the input view based on the selected model.

Model Modules

Anemia module: Predicts no anemia, mild HGB anemia, iron anemia, folate anemia, or B12 anemia from CBC and nutrition markers.

Diabetes module: Predicts no diabetes, prediabetes, or diabetes from structured health indicators.

Leukemia module: Predicts suspected leukemia from a microscopic blood smear image and returns Grad-CAM maps for each image model.

Unified interface: Provides model selection, manual entry, CSV batch analysis, OCR for anemia reports, image upload for leukemia, and result rendering.

5. Data Sources and Target Definitions

Datasets and Targets

Anemia: dataset/SKILICARSLAN_Anemia_DataSet.xlsx. Target column: All_Class. Leakage columns are removed before training.

Diabetes: dataset/diabetes_012_health_indicators_BRFSS2021.csv. Target column: Diabetes_012 with classes 0, 1, and 2.

Leukemia: lekumiai_train.ipynb builds the image dataset from C-NMC Leukemia, ALL 4-Class cancer images, WBC normal images, and Intel scene images for validator negatives. The trained weights are stored in lekumiai_model.

Stored artifacts: Tabular models are joblib bundles. Leukemia models are PyTorch state dictionaries plus evaluation figures.

6. Theoretical Methods Used

6.1 Numeric Cleaning and Leakage Removal

The tabular training scripts convert feature columns to numeric values and remove target leakage columns. This prevents the model from learning labels that would not be available at prediction time.

6.2 Median Imputation

Missing numeric values are replaced by the median value learned from the training data. Median imputation is robust to outliers and allows prediction when some user fields are empty.

6.3 Standardization

StandardScaler transforms values onto a comparable scale. Even though Random Forests are not highly scale-sensitive, a consistent preprocessing pipeline improves reproducibility and simplifies deployment.

6.4 Stratified Splitting

Stratified train-test splitting preserves class proportions in training and testing sets. This is important for medical datasets where positive cases may be rare.

6.5 Class Imbalance Handling

The anemia model uses SMOTE to synthesize minority examples. Both anemia and diabetes models use class weights to increase attention to clinically important minority classes.

6.6 Random Forest

Random Forest is an ensemble of decision trees. It captures nonlinear interactions between features and is suitable for tabular clinical data where interpretability, robustness, and low training complexity matter.

6.7 Convolutional Neural Networks

The leukemia image model uses convolutional neural networks that learn spatial patterns in cell images. MobileNetV3-Small validates whether the image is a blood smear, while EfficientNet-B5 and EfficientNet-B4 classify leukemia likelihood.

6.8 Transfer Learning

The leukemia architecture uses proven image backbones and adapts them to the blood-cell classification task. This is useful when specialized medical datasets are smaller than general image datasets.

6.9 Ensemble Prediction and Test-Time Augmentation

The leukemia probability combines B5 and B4 outputs, with a larger weight for B5. Test-time augmentation averages predictions over transformed versions of the same image, reducing sensitivity to orientation and crop variation.

6.10 Grad-CAM

Grad-CAM uses gradients in convolutional layers to highlight image regions that contributed to a model output. The application displays Grad-CAM maps for the validator, EfficientNet-B5, and EfficientNet-B4.

6.11 Leukemia Training Workflow

The leukemia notebook is not only an evaluation notebook; it contains the full training workflow used to create the deployed models. It downloads and organizes image data, builds labels, creates train/validation/test splits, defines augmentations, trains the validator and classifiers, calibrates thresholds, saves weights, and exports evaluation figures.

The final classification dataframe contains 30,536 images: 20,534 normal images and 10,002 leukemia images. The split used in the notebook is 21,375 training images, 4,580 validation images, and 4,581 test images. The source breakdown includes C-NMC leukemia images, ALL 4-Class cancer images, and WBC normal images.

A separate image validator is trained with MobileNetV3-Small to distinguish valid blood-smear images from non-medical images. Its training set combines blood-smear images with Intel scene images, so the application can reject unrelated uploads before leukemia classification.

For classifier training, images are resized and normalized, then augmented with flips, rotations, brightness and contrast changes, hue and saturation shifts, CLAHE, RGB shifts, noise, blur, geometric distortion, coarse dropout, and gamma variation. Blood-cell images also receive CLAHE plus Gaussian blur preprocessing where appropriate.

The main leukemia classifier is EfficientNet-B5 with an attention gate and a multi-layer head. A second EfficientNet-B4 classifier is trained as a complementary model. Both use transfer learning, AdamW optimization, cosine learning-rate scheduling, weighted focal loss, label smoothing, gradient clipping, automatic mixed precision, weighted sampling, early stopping, and MixUp during part of training.

After training, the notebook calibrates B5, B4, and ensemble thresholds on the validation set using F2-score maximization. The final deployed decision combines EfficientNet-B5 and EfficientNet-B4 with test-time augmentation, then produces confusion matrix, ROC curve, precision-recall curve, training history, and Grad-CAM outputs.

7. Programming and System Implementation

Main Files

`train_model.py`: Builds the anemia scikit-learn pipeline using imputation, scaling, SMOTE, and Random Forest, then saves `model/anemia_model.joblib`.

`train_diabetes_model.py`: Builds the diabetes scikit-learn pipeline using imputation, scaling, Random Forest, optional stratified sampling, and joblib export.

`lekumiai_train.ipynb`: Defines leukemia image training, preprocessing, augmentation, validator training, EfficientNet training, threshold calibration, evaluation plots, and Grad-CAM examples.

`api.py`: Implements tabular `ModelService`, OCR extraction, `LeukemiaImageService`, image inference, test-time augmentation, and Grad-CAM generation.

`web_app.py`: Runs the local HTTP server and maps model keys to their corresponding services.

web/index.html, web/app.js, web/styles.css: Implement the user interface, model switching, form generation, CSV parsing, image upload, result cards, and Grad-CAM display.

7.1 Backend API

The backend uses Python ThreadingHTTPServer. It exposes GET /api/models, GET /api/model-info, POST /api/predict, POST /api/predict-batch, POST /api/extract-from-image, and POST /api/predict-image.

7.2 Frontend

The frontend is written in plain HTML, CSS, and JavaScript. It requests model metadata and builds the correct interface: numeric forms for tabular models, OCR upload for anemia, and blood smear image upload for leukemia.

7.3 Model Serialization

The structured models are serialized as joblib bundles containing pipeline, features, class names, negative class value, and metrics. The leukemia model loads PyTorch .pth files from lekumiai_model.

8. End-to-End Workflow

-
- The user runs python web_app.py and opens the local URL.
 - The frontend loads available models and fills the model selector.
 - When a model is selected, the frontend retrieves its feature and capability metadata.
 - For anemia and diabetes, samples are collected manually or from CSV rows and sent to tabular endpoints.
 - For anemia OCR, EasyOCR reads a report image and maps detected text to known CBC marker aliases.
 - For leukemia, the uploaded image is sent to /api/predict-image as base64.
 - The backend returns predicted class, probability, per-class probabilities, quality details, and Grad-CAM explanations.
 - The frontend renders result cards and, for leukemia, model focus maps on the right side of the interface.

9. Evaluation, Explainability, and Safety

The structured models store accuracy, macro F1-score, confusion matrix, test count, and class counts. Macro F1 is important because it gives each class more equal importance than simple accuracy.

For leukemia, the final notebook test run reports Ensemble + TTA accuracy of 87.62%, recall or sensitivity of 99.86%, precision of 72.29%, F1-score of 83.87%, F2-score of 92.79%, specificity of 81.80%, AUC-ROC of 0.9500, and only 2 false negatives on the test set. The notebook also produces confusion matrix, ROC curve, precision-recall curve, training history, and Grad-CAM images. The current application computes Grad-CAM overlays dynamically for all leukemia image models.

Safety is central: the application is a decision-support prototype, not a standalone diagnostic authority. Clinical validation, external testing, bias analysis, privacy controls, and physician oversight are required before real medical use.

10. Limitations and Future Work

- The application depends on local Python and required packages such as scikit-learn, torch, timm, Pillow, and OpenCV.
- OCR extraction can fail on unclear images or unfamiliar laboratory report templates.
- The tabular models depend on the representativeness and quality of their datasets.
- Grad-CAM explains model focus but does not prove medical causality.
- Future work should add external validation, probability calibration, model versioning, automated tests, secure user management, and prediction-level PDF exports.

11. Conclusion

This project demonstrates how classical machine learning, deep learning, explainable AI, and web programming can be integrated into a single clinical classification platform. It improves over older manual or isolated workflows by providing reusable preprocessing pipelines, dynamic model selection, batch processing, image-based leukemia classification, OCR support, and visual interpretability.